High Performance Computing - MPI: Assignment 1

Name: Azzaoui Mohamed Omar
Program: Master Security IT & Big Data
Course: High Performance Computing
Professor: C. El Amrani

Exercise 1: Hello world, even and odd ranks

1. Writing the program

We started from the MPI Hello World program of the course. Each process gets its rank and the total number of processes, then prints them together with "even" or "odd" depending on its rank. Only process 0 also prints the total number of processes on a separate line.

```
└─$ cat ex1.c
#include <mpi.h>
#include <stdio.h>


int main(int argc, char *argv[]){
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("process %d of %d: Therank is %s\n", rank, size, (rank%2== 0) ? "even" :"odd");
    if (rank == 0){
        printf("total number of processes: %d\n", size);
    }
    MPI_Finalize();
    return 0;}
```

2. Compiling and running the program

We compiled the program with mpicc and ran it with mpiexec. The screenshot shows the run with 4 processes: every process printed its rank, the total of 4 and whether its rank is even or odd, and process 0 printed the extra line with the total number of processes. The runs with 8 processes are shown in the next step.

```
$ mpiexec -np 2 ./ex1
process 0 of 2: Therank is even
total number of processes: 2
process 1 of 2: Therank is odd

[~/…/MST-SIBD/S3/HPC-bigdata/assig
$ mpiexec -np 4 ./ex1
process 1 of 4: Therank is odd
process 3 of 4: Therank is odd
process 0 of 4: Therank is even
total number of processes: 4
process 2 of 4: Therank is even

[~/…/MST-SIBD/S3/HPC-bigdata/assig
$ mpiexec -np 8 ./ex1
process 1 of 8: Therank is odd
process 3 of 8: Therank is odd
process 4 of 8: Therank is even
process 7 of 8: Therank is odd
process 0 of 8: Therank is even
total number of processes: 8
process 2 of 8: Therank is even
process 5 of 8: Therank is odd
process 6 of 8: Therank is even
```

3. Why the order of the lines changes between runs

We ran the program several times. With 4 processes the order stayed the same in the three runs we tried, so we switched to 8 processes, ran the program six times in a loop, and shortened each line to its first 9 characters so that each run fits on one row and the orders are easy to compare.

```
$ for i in 1 2 3 4 5 6; do mpiexec -np 8 ./ex1 | cut -c1-9 | tr '\n' ' '; echo; done
process 0 total num process 1 process 2 process 3 process 4 process 5 process 6 process 7
process 1 process 2 process 3 process 4 process 5 process 6 process 7 process 0 total num
process 0 total num process 1 process 2 process 3 process 4 process 5 process 6 process 7
process 0 total num process 4 process 2 process 6 process 1 process 3 process 5 process 7
process 0 total num process 1 process 2 process 3 process 4 process 5 process 6 process 7
process 0 total num process 1 process 2 process 3 process 4 process 5 process 6 process 7
```

Answer: We ran the program with 8 processes six times, and the order of the lines changed from one run to another. In four runs the lines came out in order, with process 0 and its total line first, followed by processes 1 to 7. In the second run, process 0 and its total line came out last, after process 7. In the fourth run, process 0 came first, but the others appeared as 4, 2, 6, 1, 3, 5, 7. This happens because the processes run independently and at the same time. Nothing in the program synchronizes the printing, so each process prints whenever the operating system schedules it, and `mpiexec` forwards the outputs to the terminal in the order they arrive. The order is therefore not guaranteed, even when several runs look the same.

Exercise 2: Ping-pong with an integer

1. Process 0 sends the integer 100 to process 1 with tag 1

In the program, process 0 sets the integer to 100 and sends it to process 1 with tag 1.

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]){
    int rank, size, value;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0){
        value = 100;
        MPI_Send(&value, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
        printf("process 0 sent %d to process 1 with tag 1\n", value);
        MPI_Recv(&value, 1, MPI_INT, 1, 2, MPI_COMM_WORLD, &status);
        printf("process 0 received %d from process 1 with tag 2\n", value);
    }
    else if(rank==1){
        MPI_Recv(&value, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        printf("process 1 received %d from process 0\n", value);
        value = value*2;    // double it
        MPI_Send(&value, 1, MPI_INT, 0, 2, MPI_COMM_WORLD);
        printf("process 1 sent %d back to process 0 with tag 2\n", value);
    }
    // rank >= 2 does nothing

    MPI_Finalize();
    return 0;
}
```

2. Process 1 receives, doubles and sends back the integer

Process 1 receives the integer from process 0 with tag 1, displays it, doubles it and sends the result back to process 0 with tag 2.

```
┌──[~/…/MST-SIBD/S3/HPC-bigdata/assignment1-HPC]
└─$ mpiexec -np 2 ./ex2
process 0 sent 100 to process 1 with tag 1
process 0 received 200 from process 1 with tag 2
process 1 received 100 from process 0
process 1 sent 200 back to process 0 with tag 2
```

3. Process 0 receives the answer

Process 0 waits for the answer from process 1 with tag 2 and displays it. The processes with rank 2 or higher are not in any branch of the program, so they do nothing.

```
└─$ mpiexec -np 4 ./ex2
process 0 sent 100 to process 1 with tag 1
process 0 received 200 from process 1 with tag 2
process 1 received 100 from process 0
process 1 sent 200 back to process 0 with tag 2
```

When we ran the program, the lines were not printed in the order in which the events really happened: process 0 appeared to receive 200 before process 1 had sent it. Adding `fflush(stdout)` after each printf did not change this. To find the real order, we made a copy of

the program, called ex2-time, that prints a timestamp from `MPI_Wtime()` in each line, and ran it with 2 processes.

```
└─$ mpiexec -np 2 ./ex2-time
[0.000004] process 0 sent 100 to process 1 with tag 1
[0.000016] process 0 received 200 from process 1 with tag 2
[0.000006] process 1 received 100 from process 0
[0.000014] process 1 sent 200 back to process 0 with tag 2
```

The timestamps show that process 0 sent 100 (0.000004 s), process 1 received it (0.000006 s), process 1 sent 200 (0.000014 s), and process 0 received it (0.000016 s). So the program works as expected, and only the display order of the lines is misleading.

4. Running the program with only one process

With only one process, the program crashed. MPI reported "Invalid rank has value 1 but must be nonnegative and less than 1", because process 0 tried to send to process 1, which does not exist when there is a single process. To handle this case, we added a test on the number of processes: if there are fewer than 2, the program prints a message and calls `MPI_Abort`.

```
└─$ mpiexec -np 1 ./ex2
Abort(1) on node 0 (rank 0 in comm 0): application called MPI_Abort(MPI_COMM_WORLD, 1) - proce
ss 0
```

5. Process 1 waiting for tag 2 instead of tag 1

For this question we made a copy of the program, called ex2_tag, and changed only the tag in the `MPI_Recv` of process 1 from 1 to 2. The first screenshot shows the change in the code and the second one shows the run.

```
    else if(rank==1){
        MPI_Recv(&value, 1, MPI_INT, 0, 2, MPI_COMM_WORLD, &status);
```

```
└─$ mpicc ex2_tag.c -o ex2_tag

┌──[~/…/MST-SIBD/S3/HPC-bigdata/assignment1-HPC]
└─$ timeout 10 mpiexec -np 2 ./ex2_tag
process 0 sent 100 to process 1 with tag 1

=====================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   PID 145921 RUNNING AT [redacted]
=   EXIT CODE: 15
=   CLEANING UP REMAINING PROCESSES
=   YOU CAN IGNORE THE BELOW CLEANUP MESSAGES
=====================================================================================
YOUR APPLICATION TERMINATED WITH THE EXIT STRING: Terminated (signal 15)
This typically refers to a problem with your application.
Please see the FAQ page for debugging suggestions
```

Answer: If process 1 waits for tag 2 instead of tag 1, the program does not finish. Process 0 sends the message with tag 1 and prints that it sent it, but the `MPI_Recv` of process 1 only accepts a message with tag 2, so the message is not delivered to it. Process 1 stays blocked in `MPI_Recv`, and process 0 stays blocked waiting for the answer that process 1 never sends. Both processes

wait for each other, which is a deadlock. When we ran it, only the line from process 0 was printed, and we used `timeout 10` to stop it after 10 seconds, which made `mpiexec` report that the application was terminated by signal 15.

Exercise 3: Passing a token around a ring

1. Writing and running the program

We wrote the program for a ring of 4 processes. Process 0 creates the token with value 0 and sends it to process 1. Each of the other processes receives the token from the previous process, prints it, adds its own rank and sends it to the next process. Finally, process 0 receives the token back and prints the final value. Process 0 sends first and the other processes receive first, so the ring does not block. The first screenshot shows the code and the second one shows the execution with 4 processes.

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc ,char *argv[]){
        int rank,size,token;
        MPI_Status status;
        MPI_Init(&argc,&argv);
        MPI_Comm_rank(MPI_COMM_WORLD,&rank);
        MPI_Comm_size(MPI_COMM_WORLD,&size);

        int next=(rank+1)%size;
        int prev=(rank-1+size)%size;

        if (rank == 0){
                token=0;
                MPI_Send(&token,1,MPI_INT,next,1,MPI_COMM_WORLD);
                printf("process 0 created the token %d and sent it to %d\n",token,next);
                MPI_Recv(&token,1,MPI_INT,prev,1,MPI_COMM_WORLD,&status);
                printf("process 0 received the final token %d\n",token);
        }
        else{
                MPI_Recv(&token,1,MPI_INT,prev,1,MPI_COMM_WORLD,&status);
                printf("process %d received the token %d\n",rank,token);
                token+=rank;
                MPI_Send(&token,1,MPI_INT,next,1,MPI_COMM_WORLD);
                printf("process %d added its rank and sent the token %d to %d\n",rank,token,n
        }

        MPI_Finalize();
        return 0;
}
```

```
[~/…/MST-SIBD/S3/HPC-bigdata/assignment1-HPC]
$ mpirun -np 4 ./ex3-correct
process 0 created the token 0 and sent it to 1
process 0 received the final token 6
process 1 received the token 0
process 1 added its rank and sent the token 1 to 2
process 2 received the token 1
process 2 added its rank and sent the token 3 to 3
process 3 received the token 3
process 3 added its rank and sent the token 6 to 0
```

As in the previous exercise, the lines are not printed in the order of the events. For example, the final line of process 0 appears before the lines of processes 1 to 3, although it can only be printed after process 3 has sent the token. The values on the lines are still consistent with the real order, so we filled the table from them.

2. Table

Process	Receives	Adds	Sends
0	-	-	0
1	0	1	1
2	1	2	3
3	3	3	6
0 (end)	6	-	-

The "Adds" column is the rank of each process, which matches the difference between the value each process received and the value it sent.

3. Final value printed by process 0

Answer: Process 0 prints 6. The token starts at 0 and each process adds its own rank before passing it on, so the final value is 0 + 1 + 2 + 3 = 6.